

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project: Intelligent Enterprise Assistant

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements of the *Intelligent Enterprise Assistant*. The purpose of this document is to provide a clear, structured, and comprehensive description of the system's objectives, scope, constraints, and operational behavior for stakeholders, developers, and academic evaluators.

The document serves as a formal agreement on what the system will achieve and establishes a foundation for design, implementation, testing, and deployment activities.

1.2 Problem

In enterprise environments, employees frequently require access to HR policies, IT support documentation, and internal procedural guidelines. Traditional methods of retrieving such information such as manually searching lengthy PDF documents or contacting support teams are time-consuming and inefficient. This often leads to delays in task completion, reduced productivity, and increased dependency on administrative personnel.

There is a need for an intelligent system that allows employees to retrieve accurate and context-specific information quickly using natural language queries while ensuring that responses are grounded in official organizational documents.

1.3 Scope

The Intelligent Enterprise Assistant is an AI-driven system designed to provide employees with a conversational interface for querying enterprise HR and IT policy documents. The system leverages Retrieval-Augmented Generation (RAG) to combine semantic document retrieval with large language model (LLM) response generation, ensuring answers are both context-aware and document-grounded.

The scope of the system includes:

- Secure user authentication using email-based One-Time Password (OTP)
- Semantic retrieval of relevant information from official policy documents
- Natural language query handling
- Document summarization and keyword extraction for user-uploaded personal documents
- Content moderation using a predefined bad-word dictionary
- Support for a minimum of five concurrent users
- Response time under five seconds under normal operating conditions

1.2 Definitions

- **RAG (Retrieval-Augmented Generation):** A technique that enhances LLM responses by retrieving relevant context from document embeddings before generating an answer.
- **Embedding:** A numerical vector representation of text that captures semantic meaning.
- **Vector Database (FAISS):** A system used to store and search high-dimensional embeddings for similarity-based retrieval.
- **LLM (Large Language Model):** A deep learning model used to generate natural language responses.
- **OTP (One-Time Password):** A temporary authentication code sent to the user's email for verification.
- **JWT (JSON Web Token):** A token-based authentication mechanism used to manage secure user sessions.

2. System Description

2.1 User Classes

Employee: Can register via email, receive OTP, upload policy PDFs, and ask questions.

Admin (Conceptual): Uploads the Policy Documents, manages the bad-word list for content moderation.

2.2 System Architecture (RAG Flow)

The system follows a modular architecture to ensure fast response times (<5 seconds).

Ingestion: PDF → Text Extraction → Chunking → Vector Embeddings (SentenceTransformers).

Storage: Embeddings are stored in a FAISS vector index.

Retrieval: User Query → Similarity Search in FAISS → Context Retrieval.

Generation: Context + User Query → LLM API → Final Answer.

3. Functional Requirements

FR1 – OTP Authentication: System must send a 6-digit OTP to user email for secure login.

FR2 – PDF Upload: Users can upload 8–10 page PDFs for AI analysis.

FR3 – NLP Query Handling: Users can ask questions in plain English.

FR4 – Document Summarization: System must generate a summary of uploaded documents.

FR5 – Keyword Extraction: System must extract important keywords from uploaded PDFs.

FR6 – Content Moderation: System must block queries containing offensive language using a local dictionary.

FR7 – Session Management: System must manage user sessions securely using JWT.

FR8 – Concurrent Handling: System must process at least 5 user requests simultaneously.

FR9 – Response Time: System must generate responses within 5 seconds under normal load.

4. Non-Functional Requirements

- Performance: Response time must remain under 5 seconds.
- Scalability: System must handle at least 5 concurrent users.
- Security: Email OTP authentication and JWT session control must be implemented.
- Usability: Interface must be simple, intuitive, and user-friendly.
- Reliability: System must handle invalid inputs gracefully and log errors.

5. Technology Stack

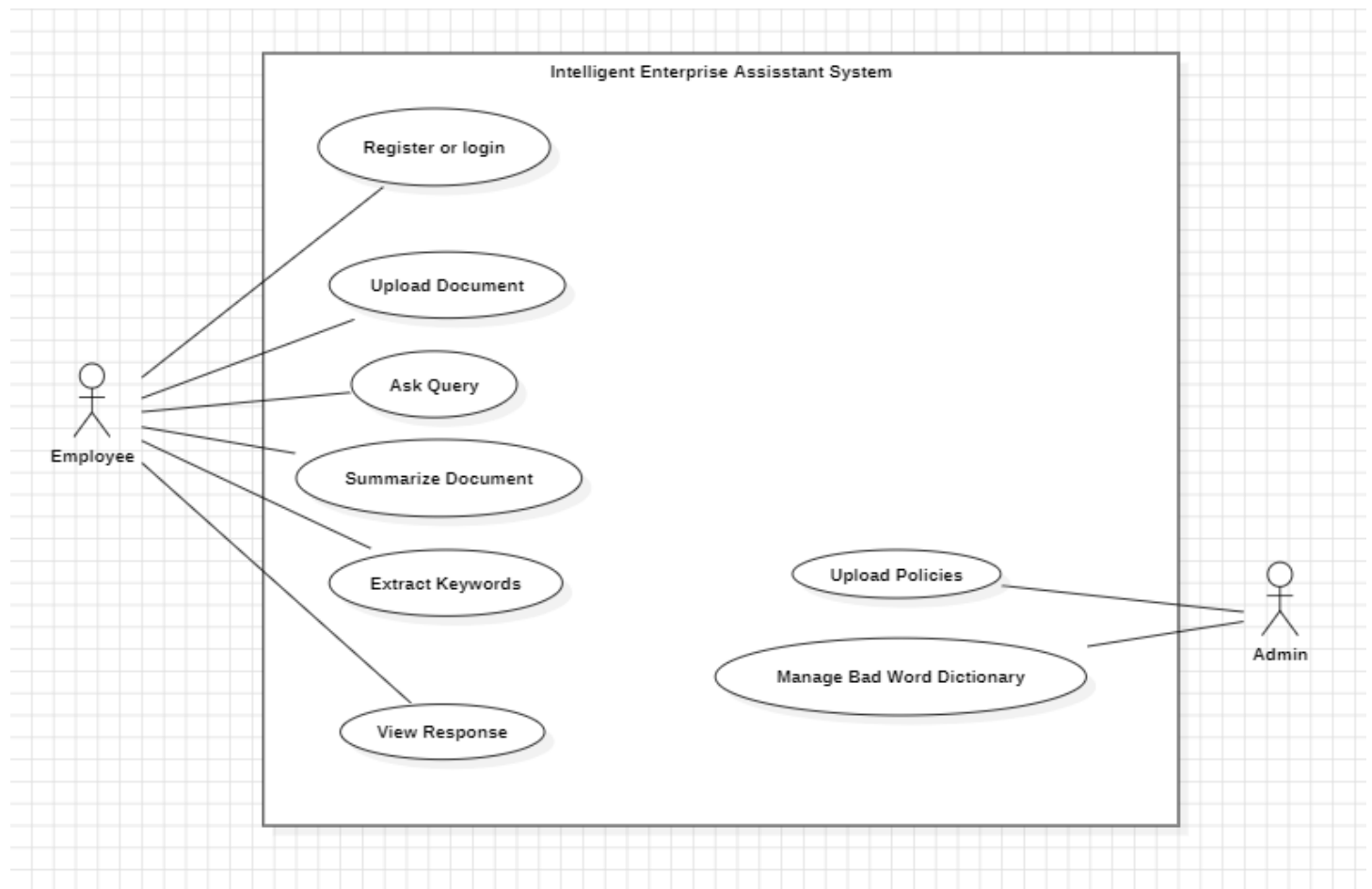
- Backend: Python, FastAPI
- AI Logic: LangChain, SentenceTransformers, HuggingFace API
- Vector Database: FAISS
- Relational Database: SQLite
- Frontend: HTML5, CSS3, JavaScript
- Authentication: SMTP (Email OTP), JWT

6. Appendices

Timeline:

Phase	Week	Activity
Phase 1	Week 1-2	Requirement analysis, vision document, use case modeling
Phase 2	Week 3-4	Static & dynamic modeling (Class, Object, Activity diagrams)
Phase 3	Week 5-7	System design (Sequence, Component, Deployment diagrams)
Phase 4	Week 8-10	Implementation (Authentication, RAG pipeline, UI integration)
Phase 5	Week 11-12	Testing, cloud deployment, final demo

Use Case Diagram:



Activity diagram:

